UNA GUIDA BM

Personaggi e modelli di gioco: sprite, pre-render 2D e 3D

Come creare i modelli di personaggi giocabili, NPC ed elementi di gioco: sprite, 2D pre-renderizzato e 3D.

Indice

Cosa copre questo manuale 5

Gli strumenti e cosa fa ciascuno 5

Da sapere 5

Attenzione 5

Scegliere una tecnica 7

Cosa fa 7

Perché conta 7

Da sapere 7

Sprite a mano 8

Cosa fa 8

Le chiamate che usa un gioco 8

Da sapere 8

Attenzione 8

Personaggi da platform (visti di lato) 9

Cosa fa 9

Cosa disegnare 9

Come lo disegna un gioco 9

Collisioni e movimento 9

Hitbox dei fotogrammi 9

Personaggi visti dall'alto, 4 direzioni 10

Cosa fa 10

Cosa disegnare 10

Da sapere 10

Personaggi visti dall'alto, 8 direzioni 11

Cosa fa 11

Lo specchio è il risparmio 11

Un modulo per tutte le animazioni 11

Dai tasti alla direzione 12

Come fare le 8 direzioni 12

Ottimizzare gli sprite . . . 13
Specchio . . . 13
Riusare parti di sprite . . . 13
Palette . . . 13
Buio e bagliore con la stessa palette . . . 14
Sprite stacking . . . 14

2D pre-renderizzato . . . 16
Cosa fa . . . 16
Come lo fanno i giochi del repository . . . 16
La stessa cosa sulla console: bm Animator F4 . . . 16
Le hitbox dalle ossa . . . 17
Ritoccare con bm Pixel . . . 17
Da sapere . . . 17
Cosa F4 non fa ancora . . . 17
Provalo, passo per passo . . . 17

Mesh . . . 19
Cosa fa . . . 19
Da un'immagine a una mesh . . . 19
Da una mesh a un'immagine . . . 19
Da mesh a modello e ritorno, con bm Mesh . . . 20
Modificare vertici e facce . . . 20
Da dove vengono le mesh del gioco . . . 20
Attenzione . . . 20

Modelli 3D . . . 21
Cosa fa . . . 21
bm Studio: il modello . . . 21
bm Animator: lo scheletro e le animazioni . . . 21
Dai un nome alle ossa pensando al gioco . . . 21
Modelli fuori dalla console . . . 21
Attenzione . . . 22

Disegnare e animare nel gioco . . . 23

Il modello e la sua animazione . . . 23
Luce, ombra, ombreggiatura . . . 23
I flag di `draw3d` . . . 24
Materiali nel colore di una faccia . . . 24
Shader . . . 24
Codice da copiare . . . 24

Tutorial . . . 26
Un eroe da platform . . . 26
Un eroe visto dall'alto con 4 direzioni . . . 26
Un personaggio 3D generato in 8 direzioni . . . 26

Consigli e limiti . . . 27
Limiti . . . 27
Consigli . . . 27
Quando qualcosa va storto . . . 27

Punti aperti . . . 28

LEGGIMI

Cos'è. La guida per creare i *modelli* di un gioco: l'eroe, i nemici, la gente del paese, gli oggetti di scena. Riunisce ciò che gli editor della suite, l'API e i giochi del repository fanno già.

Come usarla. Leggi i primi due capitoli per scegliere una tecnica, poi vai al capitolo di quella tecnica. Ciò che sta in un riquadro con bordo e ombra è fatto per essere copiato così com'è.

Cosa manca. Ciò che non esiste ancora, o esiste in parte, è segnato **[IN SVILUPPO]** con una riga su ciò che manca. Ciò che è previsto per la prima revisione beta dice "beta rev 1". Non si inventa nulla.

Cosa copre questo manuale

Un personaggio in bm è una di tre cose: uno **sprite** nello sprite sheet, un insieme di **sprite pre-renderizzati** (un modello 3D disegnato prima nello sheet) o un **modello 3D** disegnato dal vivo. Questo manuale spiega come crearli. Non spiega il codice del gioco che li usa, che sta nei manuali dell'API.

Gli strumenti e cosa fa ciascuno

Strumento	Cosa produce	Capitolo
bm Pixel	sprite, tile, palette; ritocca gli sprite generati	Sprite a mano
bm Animator, pagina F4	sprite 2D pre-renderizzati da un modello 3D, con hitbox dalle ossa	2D pre-renderizzato
bm Studio	modelli 3D da blocchi e tile; un modello da un'immagine	Modelli 3D
bm Animator, pagine F2 e F3	lo scheletro e le animazioni di un modello	Modelli 3D
bm Mesh	i vertici e le facce di un modello; le mesh costruite dal codice di un gioco	Mesh
`scripts/bmres.py` , `tools/`	estrarre, aggiungere e convertire risorse; modelli da immagini, sul PC	Mesh, 2D pre-renderizzato
bm-forge	modelli, scheletri e anteprime da riga di comando, sul PC	Modelli 3D

Da sapere

- Pixel, Studio, Animator e Mesh lavorano sullo stesso file: un **progetto** (`.bme`). Un gioco (`.bm` , `.b16`) è di sola lettura e il primo salvataggio crea una copia `.BME` modificabile.
- Lo sprite sheet è condiviso: bm Pixel lo disegna, bm Studio ne usa dei pezzi come texture, bm Animator ci mette gli sprite generati.
- Tasti e pad di ogni editor stanno nel suo manuale (`editor-pixel` , `editor-studio` , `editor-animator` , `editor-mesh`). Qui solo ciò che serve a un personaggio.

Attenzione

- Il manuale di bm Animator dice che il kernel dell'RGB30 non lo ha, quindi generare sprite sulla console RGB30 è **[IN SVILUPPO]**: oggi non è disponibile lì.

Scegliere una tecnica

Cosa fa

Lo stesso eroe si può fare in tre modi. Scegli in base all'aspetto che vuoi e a quanti fotogrammi puoi permetterti.

Tecnica	Aspetto	Costo	Adatta a
Sprite a mano	pixel art, di tua mano	ogni fotogramma lo disegni tu	platform, piccoli giochi visti dall'alto, oggetti
2D pre-renderizzato	pixel art da un modello 3D: la stessa figura da più lati, sempre in proporzione	un modello, uno scheletro, poi un solo bake	picchiaduro, giochi in vista 3/4 con 8 direzioni
Modello 3D	vero 3D, ruota e si muove liberamente	circa 1200 triangoli a scena a 60 fps	diorami, volo, camere libere

Perché conta

Le note del repository sono chiare sul limite: disegnare a mano ogni direzione di ogni azione richiede mesi ed è difficile da tenere coerente. *Yharnam* ha oltre 5.000 fotogrammi per il cacciatore e le creature. Per questo esiste il pre-render.

Da sapere

- *Titan Clash* (`carts/titan`) e *Yharnam* (`carts/yharnam`) usano sprite pre-renderizzati.
- Un modello 3D può alimentare gli altri due: lo stesso modello è un personaggio 3D e, generato in sprite, uno 2D.

Sprite a mano

Cosa fa

Uno sprite è un pezzo dello **sprite sheet**, l'unica immagine che un gioco legge. Lo sheet è diviso in celle da 8x8; uno sprite è una cella o più. bm Pixel lo disegna ingrandito, pixel per pixel, alla dimensione di 8, 16, 32, 64 o 128 pixel (**z** cambia la dimensione).

Le chiamate che usa un gioco

Chiamata	Cosa fa
`spr(n, x, y, [w, h, flip_x, flip_y])`	disegna la cella `n`, `w` per `h` celle, anche speculare
`sspr(sx, sy, sw, sh, dx, dy, [flip_x, flip_y, zoom])`	copia un rettangolo qualsiasi dello sheet, anche speculare, più grande o più piccolo con `zoom`
`zspr(name, x, y, [frame, flip_x, flip_y, zoom])`	disegna una zona con nome; senza `frame` si anima da sola
`zone(name)`	`x, y, w, h, frames, fps, slices` della zona
`zboxes(name, [frame, kind])`	le hitbox e le hurtbox dei fotogrammi della zona

Una zona è un nome per un rettangolo dello sheet, i suoi fotogrammi (riquadri della stessa dimensione affiancati su una riga) e la sua velocità. Per i fotogrammi e gli fps mostrati nell'editor vedi il manuale di bm Pixel.

Da sapere

- Il colore 0 è trasparente. In uno sheet PNG, l'alpha sotto 128 è trasparente.
- Lo sheet arriva a 4096 x 4096 pixel.
- Disegna i fotogrammi di un'animazione uno dopo l'altro, da sinistra a destra, sulla stessa riga: è così che una zona li trova.

Attenzione

- **[IN SVILUPPO]** Dare un nome a una zona in bm Pixel. Oggi le zone si scrivono in un file di testo (`name x y width height [frames [fps]]`) e si passano a `mkbm.py --sprites` . Previsto: beta rev 1.
- I fotogrammi e gli fps che imposti in bm Pixel sono ricordati per il file ma non sono scritti nella cartuccia.

DA COPIARE

```
hero_still   0  32 16 16
hero_run     16 32 16 16 4 10
```

Personaggi da platform (visti di lato)

Cosa fa

L'eroe di un platform è visto di lato. Lo disegni una volta, rivolto a destra, e il gioco lo specchia quando guarda a sinistra.

Cosa disegnare

- Una zona per azione, rivolta a destra: `hero_still`, `hero_run`, `hero_jump`. Ognuna una riga di fotogrammi uguali.
- La velocità del ciclo sta nella zona (`fps`).
- bm Pixel offre le dimensioni 8, 16, 32, 64 e 128 pixel; una dimensione multipla delle tile della mappa tiene i piedi sulle celle del terreno.

Come lo disegna un gioco

DA COPIARE

```
local facing_left = hero.vx < 0
zspr("hero_run", hero.x, hero.y, nil, facing_left)
```

La stessa chiamata con un numero di fotogramma (`zspr("hero_run", x, y, 2, facing_left)`) disegna solo quel fotogramma: usala per un salto, dove il fotogramma segue la velocità e non l'orologio.

Collisioni e movimento

- Terreno solido, piattaforme, scale, acqua e spuntoni sono **flag delle tile** (0 solido, 1 piattaforma, 2 scala, 3 acqua, 4 fa male), letti con `fget(n, f)` e `mflags(x, y, [w, h, layer])`.
- bmlib muove un corpo al posto tuo: `lib.move(b, dx, dy)` lo ferma contro le tile solide, `lib.step(b)` aggiunge la gravità.
- Il bm SDK ha un modello, **Platform 2D**: corri e salta sulla mappa, con la camera che segue.

Hitbox dei fotogrammi

Per un picchiaduro, la sezione BOXES dà a ogni fotogramma i rettangoli `hurt`, `hit` e `body`, letti con `zboxes`. Il `H:zone(...)` di bmlib li mette nel mondo con lo stesso specchio dello sprite.

ATTENZIONE

[IN SVILUPPO] Disegnare le box in bm Pixel. bm Pixel mostra le box (**j**) ma non le disegna, non le sposta e non le ridimensiona: si scrivono nel file di testo delle zone. Previsto: beta rev 1.

Personaggi visti dall'alto, 4 direzioni

Cosa fa

Visto dall'alto, il personaggio guarda in basso, in alto, a sinistra o a destra. Quattro direzioni richiedono solo tre disegni: giù, su e un lato, perché l'altro lato è il primo specchiato.

Cosa disegnare

Un gioco lo fa con sprite di 2 per 2 celle e due fotogrammi di camminata per ogni vista:

- rivolto in basso: il primo sprite del gruppo, poi il secondo fotogramma di camminata;
- rivolto in alto: lo sprite 4 celle dopo;
- rivolto di lato: lo sprite 8 celle dopo, specchiato quando cammina verso sinistra.

DA COPIARE

```
local s, flip
if math.abs(p.fx) > 0.3 then s, flip = BASE + 8 + fr * 2, p.fx < 0
elseif p.fy < 0 then s, flip = BASE + 4 + fr * 2, false
else s, flip = BASE + fr * 2, false end
spr(s, math.floor(p.x) - 8, math.floor(p.y) - 14, 2, 2, flip)
```

Da sapere

- Lo sprite è disegnato con i piedi vicino al punto `p.x, p.y`, così l'ordine di disegno per `y` funziona: ciò che sta più in basso sullo schermo sta davanti.
- L'SDK ha un modello, **Top-down 2D**: cammina tra i muri e raccogli le monete.
- Le chiamate sono le stesse del platform (`spr`, `sspr`, `zspr` con `flip_x`).

Personaggi visti dall'alto, 8 direzioni

Cosa fa

Otto direzioni (S, SE, E, NE, N, NW, W, SW) sono quelle di un gioco in vista 3/4 come *Yharnam*: il personaggio cammina e attacca in ciascuna.

Lo specchio è il risparmio

Cinque direzioni sono disegnate (S, SE, E, NE, N). Le altre tre (SW, W, NW) sono le prime specchiate. *Yharnam* lo fa per le creature, disegnate in 5 direzioni; il cacciatore ha le 8 direzioni disegnate nello sheet.

DA COPIARE

```
-- il fotogramma a schermo: la direzione disegnata, specchiata per SW W NW
function ANIM.frame(o, set, dir)
  local d = set[o.anim].d
  if #d == 8 then return d[dir + 1][o.f], false end   -- 8 disegnate: nessuno specchio
  return d[dir <= 4 and dir + 1 or 9 - dir][o.f], dir > 4
end
```

Il gioco disegna il fotogramma con `sspr(..., flip)`. Ogni fotogramma ha un'àncora (il punto tra i piedi), quindi un fotogramma specchiato si posiziona con l'àncora specchiata a sua volta: il gioco calcola `x - (width - anchor_x)` per un fotogramma capovolto.

Un modulo per tutte le animazioni

Yharnam fa partire le animazioni del cacciatore e di ogni creatura con un piccolo modulo, `ANIM` (il modulo `anim` in `carts/yharnam/main.lua`). Copialo in un gioco che ha attori con direzioni.

Un attore è una tabella con `anim` (il nome che parte), `f` (il fotogramma, 1 è il primo), `ft` (i tick da cui il fotogramma è a schermo), `newf` (vero nel tick in cui è iniziato un fotogramma) e `rev` (vero quando va all'indietro). Un set ha, per ogni nome, `t` (i tick che dura ogni fotogramma), `loop` e `d` (i fotogrammi di ogni direzione).

Chiamata	Cosa fa
`ANIM.play(o, set, name, [rev])`	fa partire un'animazione; con `rev` parte dall'ultimo fotogramma e va all'indietro
`ANIM.step(o, set, [rate], [carry])`	avanza di `rate` tick (1 se manca); dà `true` quando un'animazione che non ripete è finita; con `carry` i tick che un fotogramma non ha usato passano al successivo
`ANIM.frame(o, set, dir)`	il fotogramma da disegnare per la direzione `dir` (0 S, 1 SE ... 7 SW) e `true` quando va disegnato specchiato

DA COPIARE

```
ANIM.play(hunter, HUNT, "slash1")                  -- parte un colpo
if ANIM.step(hunter, HUNT, 1, true) then           -- una volta per tick
  ANIM.play(hunter, HUNT, "idle")                  -- il colpo è finito
end
local fr, flip = ANIM.frame(hunter, HUNT, hunter.dir)
sspr(fr[1], fr[2], fr[3], fr[4], x, y, flip)
```

Il gioco legge `newf` e `f` per sapere quando un colpo arriva a segno (il numero di fotogramma `hit` dell'animazione).

Dai tasti alla direzione

bmlib dà la direzione di un movimento: `lib.dir8(x, y)` è uno di 8 numeri (0 destra, 1 basso-destra, 2 basso ... 7 alto-destra, `nil` se non ci si muove), e `lib.DIR8[d + 1]` è il suo vettore unitario.

Come fare le 8 direzioni

- A mano: disegna 5 e specchia 3. In bm Pixel, **h** capovolge una selezione da sinistra a destra e **y** disegna in specchio; copia una direzione, incolla, capovolgi.
- Pre-renderizzate: la pagina F4 di bm Animator disegna 1, 2, 4 o 8 direzioni di un'animazione in un colpo solo (vedi "2D pre-renderizzato").

ATTENZIONE

[IN SVILUPPO] Un comando di bm Pixel che cloni le sequenze di SE, E, NE come SW, W, NW (uno specchio orizzontale che conservi la mano che regge l'arma). Oggi lo specchio lo fa il gioco quando disegna. Previsto: beta rev 1.

Ottimizzare gli sprite

Specchio

Disegna un lato, specchia l'altro al momento del disegno: `spr` , `sspr` e `zspr` accettano tutti `flip_x` . Dimezza i disegni di una vista laterale e risparmia tre direzioni su otto. Due cautele:

- un personaggio specchiato cambia mano: una spada nella destra è ora nella sinistra. Se conta, disegna a parte il lato sinistro;
- anche le box e le àncore vanno specchiate. Il `H:zone` di bmlib prende lo specchio per le box.

Riusare parti di sprite

Titan Clash disegna il robot a strati sopra una base. Lo script di build (`carts/titan/build/mkrobot.py`) tiene in uno strato proprio tutto ciò che può cambiare: piastre d'armatura, la spallina vicina (integra o crepata), i cannoni dei bracci, la spada (sulla schiena o in mano). Uno strato contiene solo i pixel che le sue parti mostrano davanti alla base, quindi base più strati è esattamente il robot con quelle parti.

Il gioco disegna una base e poi i suoi strati in ordine fisso, con la stessa àncora:

DA COPIARE

```
-- piece(): {sx, sy, w, h, dx, dy} dello sheet, àncora in (x, y)
local function piece(r, x, y, flip)
  if flip then
    sspr(r[1], r[2], r[3], r[4], x - r[5] - r[3], y + r[6], true)
  else
    sspr(r[1], r[2], r[3], r[4], x + r[5], y + r[6])
  end
end
```

Un fotogramma e molte combinazioni di equipaggiamento costano il fotogramma una volta e ogni strato una volta.

Altri modi dello stesso tipo, dalla build di *Yharnam* (`carts/yharnam/build/mkassets.py`): ogni fotogramma è ritagliato nel suo riquadro e impacchettato nello sheet, e i fotogrammi identici si tengono una volta sola.

Palette

- Lo sheet ha una palette fino a 256 colori; **F3** di bm Pixel la modifica (aggiungere, togliere, spostare, ordinare per tinta, prendere i colori usati dallo sheet).
- Uno sheet con al massimo 256 colori si salva come `SHEET8` : una palette e sequenze, invece di 4 byte a pixel. *Titan Clash* ha così uno sheet di 2048 x 3376 pixel in 1,7 MB (`--sheet8`).
- Costruisci i colori come rampe: *Titan Clash* dà a ogni materiale 5 o 6 toni, dallo scuro al chiaro, e a un materiale luminoso (visiera, sfiati) il suo.
- La console mostra 16 bit a colore (RGB565): bm Pixel mostra accanto a ogni colore come appare sulla console.
- Un secondo set di colori per il secondo giocatore (*Titan Clash* fa un'armatura cremisi per il giocatore 2) si crea quando si costruisce lo sheet. Nell'API non c'è una funzione che scambi un colore durante l'esecuzione. **[IN SVILUPPO]** uno scambio di palette a runtime: l'API non ne documenta alcuno.

Buio e bagliore con la stessa palette

La luce di *Yharnam* non ridipinge gli sprite: ogni pixel ha un livello di luce e il suo colore passa per una *tabella di sfumatura* scelta dal gioco, così le ombre diventano blu e i colori vicino a una lampada diventano arancio restando sui colori dello sheet. Le chiamate sono `fades`, `dark_begin`, `glow` e `dark_end`.

Sprite stacking

Uno stack di sprite è un oggetto pseudo-3D fatto di fette. Ogni fetta si disegna come un'immagine piatta. Le fette stanno una accanto all'altra nello sheet. Il gioco le disegna una sopra l'altra e può girare la pila con un angolo, così sembra un oggetto solido che ruota. Un albero, un barile o un'auto possono essere uno stack.

Chiamata	Cosa fa
`stack(sx, sy, w, h, n, x, y, [rz, scale, step])`	disegna `n` fette di `w` per `h` pixel che stanno una accanto all'altra nello sheet da `(sx, sy)`, da sinistra a destra. La fetta 1 è quella in fondo e si disegna per prima. `(x, y)` è il centro della fetta in fondo. Ogni fetta ruota di `rz` radianti, è `scale` volte più grande e sta `step * scale` pixel sopra quella sotto (`step` è 1 se manca). Il colore 0 è trasparente
`stackv(sx, sy, w, h, n, x, y, [rz, scale, step])`	lo stesso, ma le fette stanno una sotto l'altra nello sheet, dall'alto in basso; quella in alto è la fetta 1
`zstack(name, x, y, [rz, scale])`	disegna una zona stack dal suo nome, con `step` 1
`zone(name)`	`x, y, w, h, frames, fps, slices` di una zona; `slices` è 0 per una zona normale e da 2 a 255 per una zona stack

Una **zona stack** è una riga del file delle zone: `stack name x y w h n` per `n` fette una accanto all'altra, oppure `stackv name x y w h n` per fette una sotto l'altra, da `(x, y)` verso il basso. La fetta in fondo viene per prima. La zona ha un solo fotogramma.

- **bm Pixel, pagina F4** crea e cambia le zone `stack`, mostra le fette una per una e mostra l'intero stack che ruota. Una fetta si disegna su F1. Il capitolo "F4: the stack" del manuale di bm Pixel ha i tasti.
- **bm Animator** conserva le fette di una zona quando si rifà il bake della zona, se la nuova zona sta ancora nello sheet.
- **Da stack a mesh.** Sul PC, `python tools/stack2mesh.py SHEET.png -o OUT.bmm --size W H -n N --at X Y` legge uno stack e scrive un file `.bmm` con un modello. Ogni pixel opaco è un voxel e i voxel dello stesso colore si uniscono in box. `--dir v` serve per fette una sotto l'altra. Il limite predefinito è 4096 vertici; `--kernel-limits` ne permette 65535 per l'RGB30. Uno stack troppo grande si ferma con un errore che indica il limite.

DA COPIARE

```lua
-- un albero di 8 fette di 16x16 pixel, che ruota
local a = 0
function _update() a = a + 0.02 end
function _draw()
  cls(1)
  zstack("tree", 80, 60, a, 2)
end
```

Avvertenze

- `stack` disegna sulla CPU, legge lo sheet a ogni chiamata e non tiene memoria. La velocità sul Pi Zero W **non è verificata su hardware**.
- Una fetta che non sta tutta nello sheet viene saltata. Uno `scale` pari o minore di 0, o sopra 64, non disegna nulla.
- La pagina F4 di bm Pixel non cancella una zona stack e non crea zone `stackv`: scrivi quelle righe nel file delle zone.
- **[IN SVILUPPO]** Un pulsante in bm Pixel o bm Studio per convertire uno stack in mesh: oggi si usa il comando qui sopra. Previsto: beta rev 1.

2D pre-renderizzato

Cosa fa

Il pre-render disegna un modello 3D, posa per posa e da ogni direzione, in pixel art, una volta sola, prima che il gioco giri. Il gioco poi disegna solo 2D: costa quanto costa uno sprite, con le proporzioni e la luce di una figura 3D.

Come lo fanno i giochi del repository

- *Titan Clash*: `carts/titan/build/mkrobot.py` costruisce un robot di solidi semplici appesi a uno scheletro, renderizza ogni posa in ortogonale a 1:1 pixel e la stilizza: sfaccettature piatte in pochi toni per materiale, contorni scuri sulla silhouette e dove una parte sta davanti a un'altra, visiera e sfiati luminosi. Dà anche le hitbox e le hurtbox dalle ossa.
- *Yharnam*: `carts/yharnam/build/mkassets.py` renderizza oltre 5.000 fotogrammi (qualche secondo ciascuno) del cacciatore e delle creature, con campi di distanza con segno, e le animazioni del cacciatore come curve a 8 direzioni. Si ridisegnano solo i fotogrammi la cui posa o il cui modello è cambiato.

Questi script girano sul PC (servono numpy e Pillow); il loro risultato (lo sheet e l'elenco dei fotogrammi) è in git, quindi per costruire il gioco non servono.

La stessa cosa sulla console: bm Animator F4

La pagina **sprites** di bm Animator disegna l'animazione di un modello con il motore 3D della console dentro lo sprite sheet, tutto sulla console. Apri un modello che ha uno scheletro e almeno un'animazione (vedi "Modelli 3D"), vai su **F4**.

Riga	Valori
animation	una di quelle del modello (o la posa di riposo)
frames	da 1 a 16
size	16, 24, 32, 48, 64, 96 o 128 pixel
directions	1, 2, 4 o 8
from above	da 0 a 75 gradi, a passi di 15
camera	piatta o prospettica
light	accesa, o spenta (colori piatti)
outline	scuro o nessuno
colours	tutti, 32, 16 o 8
fill	dal 50 % al 160 % del fotogramma
hitboxes	dalle ossa, o nessuna

[DA COMPLETARE: shots/editor-animator-sprites.png]

Invio disegna gli sprite e li mette nello sheet, sotto ciò che c'è già, una riga di fotogrammi per direzione: il fotogramma `f` della direzione `d` sta in `x0 + f * size`, `y0 + d * size`. La pagina aggiunge

una zona chiamata `model_animation` in minuscolo e mostra la chiamata `sspr()` per le altre direzioni.

Le hitbox dalle ossa

Con **hitboxes** attive, ogni osso diventa un rettangolo in ogni fotogramma. Il nome dell'osso decide il tipo: `hit`, `weapon`, `punch`, `fist`, `sword`, `blade`, `spear` o `attack` nel nome dà una box hit (rossa); `root`, `body`, `hips` o `pelvis` una box body (verde); tutto il resto una box hurt (blu).

Ritoccare con bm Pixel

I fotogrammi generati sono sprite normali nello sheet. Apri il progetto in bm Pixel, vai alla zona (il pannello dice `zone "name" frame n/m`), e correggi un pixel, aggiungi una luce o capovolgi un fotogramma. L'anteprima delle box (**j**) mostra le box sopra il tuo disegno. Se generi di nuovo, la zona con lo stesso nome viene sostituita, quindi ritocca per ultimo.

Da sapere

- Il totale di un bake deve restare sotto 512 x 512 pixel (fotogrammi per dimensione per direzioni per dimensione). Altrimenti: `too many, fewer frames or a smaller size`.
- `zspr` disegna la prima direzione. Per le altre usa `sspr()` con i numeri della riga in fondo a F4.
- Luce spenta e 16 o 8 colori danno un aspetto disegnato a mano.

Cosa F4 non fa ancora

ATTENZIONE

[IN SVILUPPO] Zone con un suffisso di direzione. F4 crea una zona per animazione, con le direzioni in righe; il progetto vuole nomi come `hunter_walk_s`, `hunter_walk_se` e un'àncora a terra sul piede. Previsto: beta rev 1.

[IN SVILUPPO] Punti di aggancio (socket). La posizione 2D di una mano o di una volata non è esportata con gli sprite (in 3D usa `bone3d()`). Previsto: beta rev 1.

[IN SVILUPPO] Strati. Un bake è un blocco unico: niente strati separati per armatura e arma come in *Titan Clash* (il progetto lo chiama bake modulare). Previsto: beta rev 1.

[IN SVILUPPO] Marcatori di evento sulla timeline (un fotogramma di colpo, un suono): il gioco conta da sé i fotogrammi. Previsto: beta rev 1.

Provalo, passo per passo

1. Apri la scheda **Dev** e avvia **bm Animator**. Scegli un progetto con un modello con scheletro (`carts/overbit` ha dei modelli con scheletro).
2. Premi **F4**. Scegli l'animazione `walk` nella prima riga.
3. Imposta `frames` a 8, `size` a 48, `directions` a 4.
4. Controlla che `hitboxes` dica `from bones`.
5. Premi **Invio** e aspetta `N sprites ... put in sheet`.
6. Premi **Ctrl+S**.
7. Apri il progetto in bm Pixel, premi **F2**, guarda le nuove righe e correggi ciò che vuoi.

PROVA

Rinomina un osso in `sword` in F2 di bm Animator, genera di nuovo e guarda la sua box diventare rossa.

Mesh

Cosa fa

Una mesh è una forma 3D fatta di punti (vertici) e triangoli (facce). In bm ce ne sono di tre tipi, e passare dall'uno all'altro è il modo in cui le forme di un gioco diventano modelli che puoi modificare.

Lettera	Cos'è	Nel gioco
M	un modello della sezione MESH (quelli di bm Studio)	`model("name")`
C	una mesh nel codice, scritta da bm Mesh come `mesh_name()` tra `-- [bm Mesh begin]` e `-- [bm Mesh end]`	`local m = mesh_name()`
G	una mesh che il codice costruisce con `mesh()`, `mesh_sphere()` o `mesh_cube()`	il codice del gioco

Da un'immagine a una mesh

Un'immagine diventa un modello in tre modi.

- **In bm Studio, nella pagina dei modelli, `m` ** (o *Model from picture...* nel menu): un'immagine dalla cartella `pics/` della SD o dalla radice della SD (`.png` o `.jpg`). **Cutout** è una figura di carta spessa con l'immagine davanti e specchiata dietro; **lathe** ruota il mezzo contorno attorno all'asse verticale (vasi, torri); **meshy.ai** è un servizio online che richiede la rete e un account.
- **Sul PC, `tools/cutout2mesh.py` **: lo stesso cutout e lathe senza rete e senza IA. In una cartuccia nuova l'immagine va sullo sheet; in una esistente le facce prendono colori piatti.
- **Sul PC, `tools/img2mesh.py` **: un modello di visione (servono l'API Anthropic e una chiave fuori dal repository) guarda l'immagine, scrive uno script di parti, e lo script viene costruito, disegnato e corretto per qualche giro. Il risultato è un `.bm` con un modello, con ossa e animazioni se lo script le ha.

DA COPIARE

```
python3 tools/cutout2mesh.py hero.png -o hero.bm --name hero
python3 tools/cutout2mesh.py vase.png -o vase.bm --lathe
```

Da una mesh a un'immagine

Un modello diventa un'immagine nei modi del capitolo "2D pre-renderizzato" (bm Animator F4). Sul PC, bm-forge disegna anteprime:

DA COPIARE

```
python .claude/skills/bm-forge/scripts/bm_forge.py preview --help
```

[DA COMPLETARE: le opzioni di `preview` per una singola immagine della dimensione di uno sprite; le note di bm-forge elencano `--turntable`, `--clip`, `--mask` e `--compare`, e dicono che un render è un PNG o una GIF]

Da mesh a modello e ritorno, con bm Mesh

Nella pagina **list** di bm Mesh:

Tasto	Cosa fa
m	copia come modello (da mesh a modello): una mesh G o C diventa un modello M che puoi nominare e salvare
c	copia come codice (da modello a `mesh_name()`): il modello diventa una mesh C
r	rinomina
d	duplica (un modello mantiene lo scheletro)
Invio	modificala

Modificare una mesh G ne fa prima una copia come modello: il codice del gioco non viene riscritto. Una copia come codice non mantiene lo scheletro.

Modificare vertici e facce

Nella pagina **edit** (F2) **Tab** passa tra vertici e facce.

Tasto	Cosa fa
g, r, t	sposta, ruota, scala la scelta
x	estrudi le facce scelte
m, M	specchia la scelta / copiala dall'altra parte dello 0
j	crea una faccia su 3 o 4 vertici scelti
k, K	unisci i vertici / salda quelli nello stesso punto
u	dividi ogni faccia scelta in 4
i	rovescia le facce
p, o, c	dipingi, prendi un colore, palette

Usa **M** per costruire un personaggio simmetrico: modella metà, copiala dall'altra parte.

Da dove vengono le mesh del gioco

`cart_meshes(path)` esegue il codice del gioco a parte, in uno stato Lua suo, una volta con `_init`, `_update` e `_draw`, e raccoglie ciò che costruiscono `mesh()`, `mesh_sphere()` e `mesh_cube()`; il nome di una mesh è il nome della variabile che la contiene. Una mesh costruita solo dopo un input può non essere trovata.

Attenzione

- Al massimo 65535 vertici e 65535 triangoli in una mesh, e 256 modelli in una cartuccia.
- bm Mesh mantiene lo scheletro di un modello: l'osso di ogni nuovo vertice è quello da cui nasce.
- Uno stack di sprite diventa un mesh con `tools/stack2mesh.py` (capitolo "Sprite stacking"). Un pulsante per farlo in bm Studio o bm Mesh non c'è ancora.

Modelli 3D

Cosa fa

Per un personaggio che nel gioco è 3D, la catena è: **modello** (bm Studio) → **scheletro** e **animazioni** (bm Animator) → il gioco lo disegna con `model()`, `animate()` e `draw3d()`.

bm Studio: il modello

bm Studio costruisce modelli con blocchi e tile, sulla console.

- Strumenti: **1** blocco (un cubo), **2** tile (un lato di una cella: pavimenti, muri, porte), **3** seleziona (sposta, ruota, capovolgi, specchia, copia facce), **4** vertice (sposta gli angoli: tetti, rampe, forme libere), **5** dipingi (pixel della tile di una faccia).
- Le texture sono pezzi dello sprite sheet del progetto. Non c'è colore agli angoli: una faccia è o una tile o un colore piatto.
- **F2** è la pagina dei modelli: nuovo, rinomina (fino a 16 lettere), duplica, elimina, riduci i triangoli (`-`), modello da un'immagine (`m`).
- La console disegna circa 1200 triangoli a scena a 60 fps; bm Studio avvisa oltre quel numero con `heavy for 60 fps`. **-** riduce un modello, mantenendo i bordi, le linee di colore e le giunzioni delle texture; lo scheletro segue.
- Un modello ha al massimo 4096 vertici e 16384 triangoli quando è salvato da bm Studio.

bm Animator: lo scheletro e le animazioni

- **F2 rig**: **n** crea lo scheletro (l'osso `root`) e aggiunge ossa; **m** specchia un osso e i suoi figli, con nomi che finiscono in `.L` e `.R`; **k** dà ogni faccia all'osso più vicino (rigido, adatto ai robot), **K** ogni angolo (morbido); **v** sceglie le facce a mano. Al massimo 64 ossa.
- **F3 animate**: keyframe su una timeline a 12 fotogrammi al secondo; ogni rotazione di un osso è un keyframe; **i** sceglie lineare, morbida o a gradino; **m** specchia una posa, quindi una camminata richiede due pose e un ciclo. Al massimo 255 animazioni per modello.
- **F1 play**: controlla l'animazione, fondila con la successiva (`b`), mostra lo scheletro (`k`).
- Lo skinning è rigido: ogni angolo segue un osso, senza pesi misti.

Dai un nome alle ossa pensando al gioco

I nomi hanno un uso: `.L` e `.R` per i due lati; `sword`, `fist` e simili per le box di un attacco; `root`, `body`, `hips` per il corpo; `head` per un colpo alla testa con `hit3d`.

Modelli fuori dalla console

- **bm-forge** (`.claude/skills/bm-forge`): costruisce modelli MESH da JSON di box o da `.glb` senza texture, scheletri e clip da JSON, ed esporta `.bm` (Pi) e `.b16` (RGB30). Una persona o un personaggio è una pelle continua con vertici condivisi alle giunture, con scheletro, mai box separate. Il modello `examples/charkit/` ha due preset.
- Un `.glb` locale della scheda SD non si apre ancora negli editor; l'importazione nella suite è prevista (APP-9). Oggi la console riceve un `.glb` solo dal servizio da immagine a 3D (`picture3d`), e bm Studio lo prende poi come modello. Sul PC, `python3 scripts/bmres.py convert IN.glb OUT.bmm` converte un `.glb` in un `.bmm`; `tools/meshy2mesh.py`, `tools/local2mesh.py` e `tools/cutout2mesh.py` fanno modelli da altre fonti. Per un gioco del repository, la build legge

`assets/models.bm` oppure `assets/models.glb` (prima `models.bm`) e mette i modelli nel `.bm`. `carts/village/assets` contiene `models.bm`, non un `.glb`.

- **[IN SVILUPPO]** Salvare e aprire i file di modello `.bmm` dal bm Studio della console. Oggi il programma salva solo il progetto; sul PC `scripts/bmres.py extract GAME.bm --model villager -o VILLAGER.bmm` ne crea uno.

Attenzione

- Un modello non ha scheletro finché bm Animator non ne crea uno; `animate()` dà un errore se il modello non ne ha.
- **[DA COMPLETARE: la dimensione di un modello che il Pi Zero W disegna fluido; le note danno i 1200 triangoli a scena come cifra per 60 fotogrammi al secondo]**

Disegnare e animare nel gioco

Tutte queste chiamate stanno nell'API 3D (`docs/api/light3d.md`).

Il modello e la sua animazione

Chiamata	Cosa fa
`models()`	i nomi dei modelli della cartuccia
`model(name)`	il modello come mesh, con la texture dello sheet; fallo in `_init`
`clips(m)`	le animazioni di un modello: `{ {name=, length=, loop=}, ... }`
`animate(m, [anim, t, anim2, t2, k, bone])`	mette il modello nella posa di `anim` al tempo `t` in secondi; con `anim2, t2, k` fonde due animazioni; con `bone` la seconda vale solo per quell'osso e quelli sotto (un busto che spara su gambe che corrono)
`bone_turn(m, bone, [rx, ry, rz])`	ruota un osso sopra l'animazione: la mira
`bones3d(m)`	i nomi delle ossa
`bone3d(m, bone)`	dove stanno testa e coda di un osso nell'ultima posa: mani, la punta di una spada, una luce
`hit3d(m, x, y, z, ry, scale, ox, oy, oz, dx, dy, dz, [maxd])`	un raggio contro le ossa nell'ultima posa
`bounds3d(m)`	il box attorno alla mesh
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	disegna il modello in posa

`animate` costa solo le ossa. Chiamala prima di `draw3d` , a ogni fotogramma.

Luce, ombra, ombreggiatura

Chiamata	Cosa fa
`camera3d(x, y, z, [yaw, pitch, fov, roll])`	la camera
`light3d(x, y, z, [ambient])`	la direzione della luce e la luce ambiente
`sky3d(sun, sky, ground)`	il colore del sole e la luce ambiente dall'alto e dal basso
`shine3d(spec, exponent, rim)`	riflessi sulle facce lucide e luce sul bordo
`lamp3d(i, x, y, z, radius, [k, colour])`	una delle 4 luci puntiformi
`fog3d(colour, near, far)`	la nebbia
`shadow3d(style)`	le ombre di `draw3d` con flag 8: 0 scurisce, 1 nero retinato

Chiamata	Cosa fa
`zclear()`	svuota lo z-buffer, a ogni fotogramma prima di `draw3d`
`cel3d([bands, floor, outline_rgb, outline_width])`	luce cel per le chiamate `draw3d` con flag 256 (da 2 a 8 bande, spenta di default) e colore e spessore del contorno per il flag 512; senza argomenti restituisce i valori

I flag di `draw3d`

Flag	Effetto
1	senza z-buffer
2	senza luce (colori piatti)
4	liscio (Gouraud)
8	l'ombra della mesh sul piano `y`, lungo il sole (la mesh non viene disegnata)
16, 32, 48	livello di dettaglio 2, 1, 0
64	davanti (braccia e armi in prima persona)
128	texture piatte (il texel medio)
256	luce cel: la luce del sole di questo disegno è tagliata in bande (vedi `cel3d`)
512	contorno della mesh, dopo di essa (vedi `cel3d`)

Materiali nel colore di una faccia

Il colore di una faccia può portare dei **bit di materiale**: bit 30 `0x40000000` emissivo (visiere, luci), bit 29 `0x20000000` lucido, bit 28 `0x10000000` a retino (scudi, vetro), bit 27 `0x08000000` piatto, bit da 24 a 26 il livello di dettaglio.

Shader

Il driver V3D del Pi esegue shader QPU scritti in `tools/qpuasm.py` e generati in `src/gpu/shaders.h`; il Mali dell'RGB30 ha un suo bring-up (`docs/progress/graphics-mali.md`). Un gioco non li scrive: sceglie con `gpu3d()` (acceso, anti-aliasing 4x, dove il vertex shader colloca i vertici) e con le righe di Settings. Le stesse funzioni disegnano sull'ARM quando la GPU è spenta.

> ATTENZIONE
> **[IN SVILUPPO]** Shader scritti dal gioco. L'API non documenta alcuna chiamata per dare a un modello uno shader proprio.

Codice da copiare

DA COPIARE

```
local v, t = nil, 0
function _init()
  v = model("villager")
end

function _update()
  t = t + 1 / 60
end

function _draw()
  cls(1)
  zclear()
  camera3d(0, 1, -4)
  light3d(-0.4, 0.8, -0.5, 0.4)
  animate(v, "walk", t)
  draw3d(v, 0, 0, 0, 0, 0, 0, 1)
  draw3d(v, 0, 0, 0, 0, 0, 0, 1, 8)
end
```

L'ultima riga disegna l'ombra. È lo schema dei giochi con modelli con scheletro, come `carts/overbit` .

Tutorial

Un eroe da platform

1. In bm Pixel premi **z** finché la dimensione è 16x16. Disegna l'eroe rivolto a destra nel primo sprite.
2. Premi **PgDn** e disegna il secondo fotogramma della corsa, poi il terzo e il quarto, uno a destra dell'altro.
3. Premi **+** finché il pannello dice 4 fotogrammi e **>** o **<** per la velocità. **p** riproduce.
4. Salva con **Ctrl+S**.
5. Scrivi la zona in `zones.txt` (`hero_run 16 32 16 16 4 10`) e costruisci con `mkbm.py --sprites zones.txt` .
6. Nel gioco: `zspr("hero_run", x, y, nil, facing_left)` .

Un eroe visto dall'alto con 4 direzioni

1. Disegna sprite di 2x2 celle (dimensione 16): rivolto in basso (due fotogrammi di camminata), rivolto in alto (due), rivolto a destra (due).
2. Mettili a 4 celle di distanza sullo sheet, come nel codice di "Personaggi visti dall'alto, 4 direzioni".
3. Disegna con `spr(s, x - 8, y - 14, 2, 2, flip)` e `flip` vero quando cammina a sinistra.

Un personaggio 3D generato in 8 direzioni

1. Costruisci il modello in bm Studio (strumenti da 1 a 5), al massimo 1200 triangoli.
2. In bm Animator F2 crea lo scheletro e chiama `sword` l'osso dell'arma; premi **k**.
3. In F3 crea `idle` e `walk` (due pose e un ciclo).
4. In F4 imposta `frames` 8, `size` 48, `directions` 8, `hitboxes` dalle ossa; premi **Invio**.
5. Apri il progetto in bm Pixel e ritocca la zona.
6. Nel gioco: `zspr("hero_walk", x, y)` per la prima direzione e `sspr()` per le altre, con i numeri mostrati in F4. Specchia le tre direzioni di sinistra come in "Personaggi visti dall'alto, 8 direzioni".

Consigli e limiti

Limiti

Cosa	Limite
dimensioni degli sprite in bm Pixel	8, 16, 32, 64, 128 pixel
fotogrammi di una zona in bm Pixel	da 1 a 16, velocità da 1 a 30 fps
palette	da 1 a 256 colori
sprite sheet	4096 x 4096 pixel
un bake (F4)	512 x 512 pixel in tutto, sprite da 16 a 128 pixel, da 1 a 16 fotogrammi
ossa in uno scheletro	64
animazioni di un modello	255
triangoli a scena a 60 fps	circa 1200
luci puntiformi (`lamp3d`)	4
modelli in una cartuccia	256

Consigli

- Scegli prima l'aspetto: a mano, pre-renderizzato o 3D. Cambiare dopo vuol dire disegnare di nuovo.
- Specchia ciò che puoi, e tieni l'àncora di ogni fotogramma ai piedi.
- Tieni i fotogrammi di un'animazione uno accanto all'altro, su una riga.
- Dai alle ossa i nomi delle box che faranno.
- Genera per ultimo, ritocca dopo il bake, e tieni una copia del progetto prima di generare di nuovo.
- Prova spesso il gioco con **F5**: salva e torna alla stessa pagina.

Quando qualcosa va storto

- `too many, fewer frames or a smaller size` : il bake supera 512 x 512 pixel; riduci i fotogrammi, la dimensione o le direzioni.
- `NO SKELETON` in bm Animator: creane uno in F2.
- `animate` dà un errore: il modello non ha scheletro o il nome dell'animazione è sbagliato; controlla con `clips(m)` .
- Lo sprite specchiato è spostato di lato: l'àncora non è stata specchiata insieme al fotogramma.

Punti aperti

- **2D pre-renderizzato**: immagine da fare: shots/editor-animator-sprites.png
- **Mesh**: le opzioni di `preview` per una singola immagine della dimensione di uno sprite; le note di bm-forge elencano `--turntable`, `--clip`, `--mask` e `--compare`, e dicono che un render è un PNG o una GIF
- **Modelli 3D**: la dimensione di un modello che il Pi Zero W disegna fluido; le note danno i 1200 triangoli a scena come cifra per 60 fotogrammi al secondo